

A PROJECT REPORT ON

SURPLUS FOOD DONATION AND WASTE REDUCTION PLATFORM

Domain: Social Impact / Web and Mobile

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Food waste is a serious problem worldwide. The UNEP Food Waste Index Report estimates that a large share of food available to consumers is wasted at the household, retail and food service levels, while millions of people in India still go to bed hungry. Hotels, wedding halls, college canteens, hostels and corporate cafeterias regularly prepare more food than is consumed, and the surplus is often thrown away at the end of the day.

Many donors are willing to give surplus food, and many NGOs, orphanages, old age homes and volunteers are ready to distribute it. However, there is no common real-time channel connecting them. By the time a donor finds someone to collect the food through phone calls, it may have spoiled. Food safety time limits are not tracked, and there is no record of how much food was saved.

This project, AnnaShare, is a web and mobile platform where donors post surplus food with quantity, type and expiry time, and nearby verified NGOs and volunteers are notified to claim it. A routing module assigns the nearest available volunteer, delivery is confirmed with a photo or OTP, and impact statistics are maintained. In addition, a simple time-series demand model forecasts the expected daily surplus for regular donors such as canteens, helping them prepare food more accurately and reduce waste at the source.

LIMITATIONS

1. Large quantities of edible food are wasted while people go hungry.
2. Donors and NGOs have no common real-time channel.
3. Food safety time limits are not tracked.
4. There is no measurement of food saved or waste reduced.

1.1 History of Domain

Food banks began in the United States in 1967 with St. Mary's Food Bank in Phoenix, collecting surplus food for distribution. In India, organisations such as the Robin

Hood Army (2014), Feeding India (2014) and the 'Roti Bank' initiatives began collecting surplus food from restaurants and events using volunteers coordinated over social media.

The Food Safety and Standards Authority of India (FSSAI) launched the Save Food Share Food initiative and issued the Food Safety and Standards (Recovery and Distribution of Surplus Food) Regulations, 2019, which provide guidelines for safe food donation. Mobile apps such as OLIO (UK) and Too Good To Go demonstrated that technology can connect surplus food with people in real time.

1.2 Overview of the Project

A donor (hotel, hall, canteen or household) posts surplus food with a photo, food type (veg/non-veg, cooked/packaged), quantity in plates or kilograms, preparation time and safe-until time. The system validates that the remaining safe window is enough for pickup and distribution (for cooked food, typically within a few hours of preparation as per safety guidance).

NGOs and volunteers within a configurable radius are found using geospatial queries in Firebase (geohash) and notified through push notifications. The first responder claims the donation. The pickup route is computed with Google Maps Directions. Delivery is confirmed with a photo and an OTP given by the receiving centre. Impact statistics such as meals saved and CO2 avoided are updated. For regular donors, a Prophet model forecasts daily surplus based on day of week, events and holidays.

1.3 Motivation

Every day, good food is thrown away a few kilometres from people who need it, simply because nobody connected them in time.

- **Motivation:** To connect surplus food with people in need before it spoils, and to help kitchens reduce surplus in the first place.
- **Outcome:** More meals reaching the needy, less food in landfills, and data-driven food preparation.

1.4 Aim of the Project

The project is entitled as “**SURPLUS FOOD DONATION AND WASTE REDUCTION PLATFORM**”. The aim of the project is to develop a real-time platform that connects food donors with nearby NGOs and volunteers, ensures safe and timely pickup and delivery, tracks impact, and forecasts surplus to reduce waste at the source.

1.5 Objectives

- To allow donors to post surplus food with safety time windows.
- To notify nearby NGOs and volunteers and allow first-come claiming.
- To compute pickup and delivery routes using maps.
- To confirm deliveries with photo and OTP.
- To forecast daily surplus using time-series models.

1.6 Literature Review

UNEP Food Waste Index Report (2021, 2024): Estimated global food waste at retail, food service and household levels and found that household food waste is substantial in all income groups, highlighting food waste as a global problem.

FSSAI (2019): The Food Safety and Standards (Recovery and Distribution of Surplus Food) Regulations provide rules for food recovery agencies on hygiene, temperature and time limits for distributing surplus food.

Taylor and Letham (2018): Described Prophet, a modular regression model for forecasting at scale with interpretable parameters for trend, seasonality and holidays.

Box and Jenkins (1970): Introduced the ARIMA methodology for time-series analysis and forecasting, widely used as a forecasting baseline.

Harvey et al. (2020): Studied the OLIO food sharing app and found that a large proportion of listed food items were successfully shared, demonstrating the effectiveness of app-based food redistribution.

Dijkstra (1959): Proposed an algorithm for finding the shortest paths between nodes in a graph, the basis for route computation in mapping services.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

Currently, donors call NGOs or volunteers they know personally, or post in WhatsApp groups. NGOs collect food when possible. There is no real-time matching, no safety time tracking and no record of impact. Canteens prepare food based on rough estimates.

DISADVANTAGES

- Delays in finding a collector; food spoils.
- Dependence on personal contacts.
- No tracking of food safety windows.
- No verification of delivery.
- No data to reduce surplus at the source.

2.2 Proposed System

The proposed AnnaShare platform allows donors to post surplus food, automatically notifies nearby verified NGOs and volunteers, assigns pickups with routes, confirms delivery with photo/OTP, tracks food safety time windows and impact, and forecasts surplus for regular donors.

ADVANTAGES

- Real-time matching within minutes.
- Food safety windows enforced.
- Verified NGOs and confirmed deliveries.
- Impact statistics motivate donors.
- Forecasting reduces over-preparation.

2.3 Project Schedule

The development of AnnaShare follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the AnnaShare system are listed below.

- **Registration and verification:** Donors, NGOs and volunteers register; NGOs verified by admin.
- **Post donation:** Donor enters food details, quantity, photo and safe-until time.
- **Safety validation:** Reject posts with insufficient safe time.
- **Nearby matching:** Find NGOs/volunteers within radius and notify.
- **Claiming:** First responder claims; others notified it is taken.
- **Routing:** Show pickup and delivery route.
- **Delivery confirmation:** Photo and OTP confirmation.
- **Impact and forecast:** Meals saved statistics and surplus forecast.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Timeliness:** Notifications within 10 seconds of posting.
- **Reliability:** Re-notify wider radius if not claimed in 15 minutes.

- **Security:** Phone OTP login; only verified NGOs can claim.
- **Usability:** Post a donation in under one minute.
- **Scalability:** Firebase scales to thousands of users.
- **Accountability:** Full audit trail of each donation.

3.3 User Requirements

- Donors post food quickly from the phone
- NGOs get instant alerts for nearby food
- Volunteers see clear pickup and drop routes
- Everyone sees the impact created
- Canteens get surplus forecasts

3.4 System Requirements

- Flutter mobile app / React web app
- Node.js Cloud Functions
- Firebase Firestore, Auth, Cloud Messaging and Storage
- Google Maps API
- Python forecasting service (Prophet)

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above
Hard Disk : 256 GB SSD
RAM : 8 GB
Mobile : Android smartphone with GPS and camera
Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Windows 10 / Ubuntu 22.04
Mobile App : Flutter (Dart)
Web App : React JS
Back End : Node.js (Firebase Cloud Functions)
Database : Firebase Firestore
Maps : Google Maps / Directions API
Forecasting : Python, Prophet / ARIMA

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed AnnaShare system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Firebase provides authentication, real-time database, push notifications and storage without managing servers. Flutter allows one code base for Android and iOS. Prophet is a free forecasting library. The project is technically feasible.

ECONOMIC FEASIBILITY

The Firebase free tier and Google Maps free credits are sufficient for a city-level pilot. NGOs and volunteers use their own phones. It is economically feasible.

OPERATIONAL FEASIBILITY

Posting and claiming are simple, quick actions. NGOs already coordinate volunteers; the app makes it faster. It is operationally feasible.

5.2 About the Software

FLUTTER

Flutter is an open-source UI software development kit created by Google. It is used to build natively compiled applications for Android, iOS, web and desktop from a single code base written in the Dart language.

Flutter's hot reload feature and rich widget library speed up mobile development, and plugins are available for GPS location, sensors, maps and Firebase.

- Single code base for Android and iOS
- Fast development with hot reload

- Native performance

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

NODE.JS AND EXPRESS

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive real-time applications.

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building REST APIs, including routing, middleware and request/response handling. Together they allow both front end and back end to be written in JavaScript.

- Handles many concurrent connections
- npm - the largest package registry
- Ideal for real-time and IoT back ends

FIREBASE

Firebase is a Backend-as-a-Service platform from Google. It provides Cloud Firestore (a real-time NoSQL database), Authentication, Cloud Storage, Cloud Messaging

(push notifications) and Cloud Functions.

Firestore pushes data changes to all connected clients instantly, which is ideal for live tracking and real-time notifications without writing a separate socket server.

- Real-time database synchronisation
- Phone OTP authentication
- Free Spark plan for prototypes

GOOGLE MAPS PLATFORM

Google Maps Platform provides APIs for embedding maps, geocoding addresses, calculating distances and generating driving or walking directions.

The Directions and Distance Matrix APIs are used to find the nearest volunteer or police station and to plan routes.

PROPHET / ARIMA

Prophet is an open-source forecasting procedure developed by Meta. It fits an additive model in which non-linear trends are combined with yearly, weekly and daily seasonality plus holiday effects. It works well with time series that have strong seasonal effects and several seasons of historical data.

ARIMA (AutoRegressive Integrated Moving Average) is a classical statistical model available in the statsmodels library and is used as a baseline for comparison.

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI),

which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of AnnaShare is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

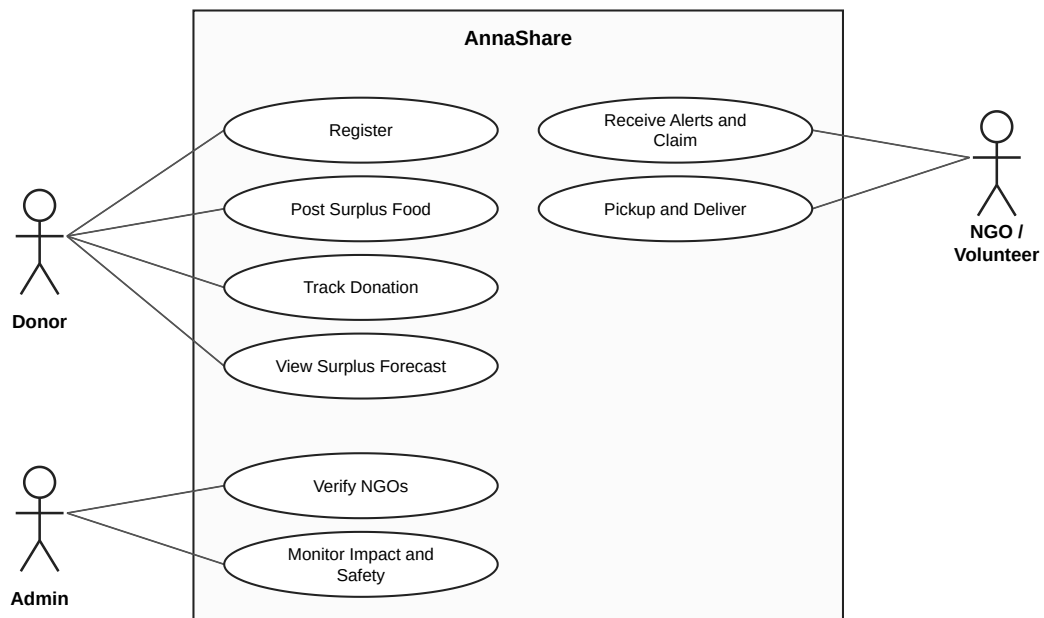


Fig 1: Use case diagram of AnnaShare

6.2 System Architecture

AnnaShare uses a serverless architecture on Firebase. Cloud Functions react to new donation documents to run matching and notifications, and a separate Python service generates surplus forecasts nightly.

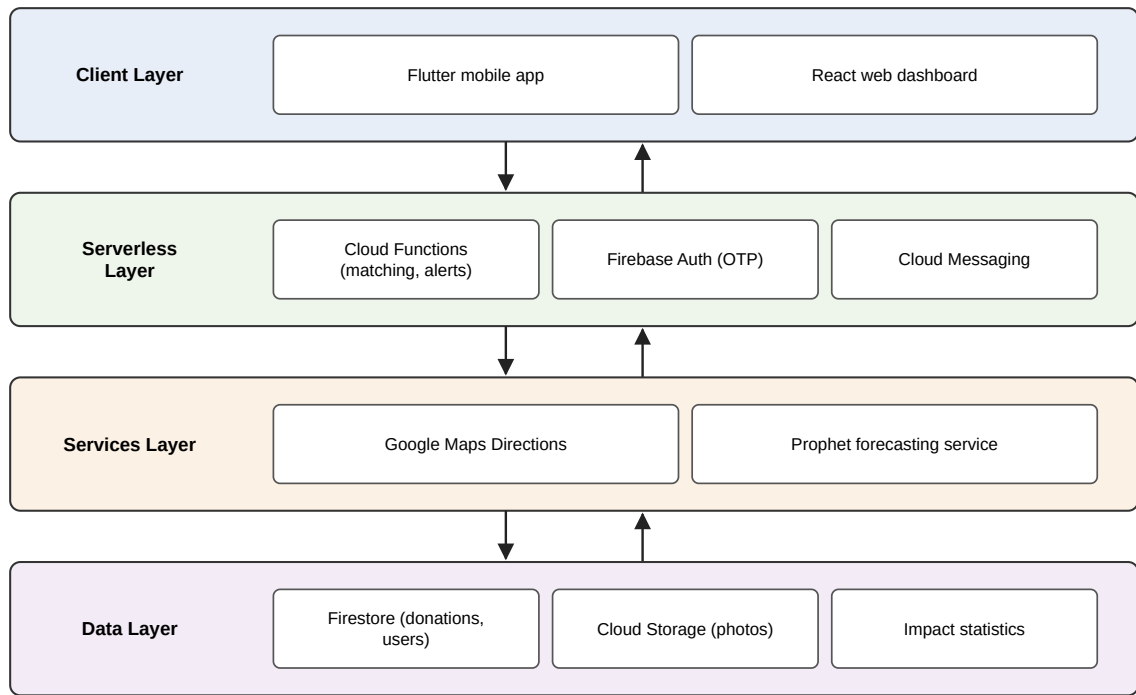


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

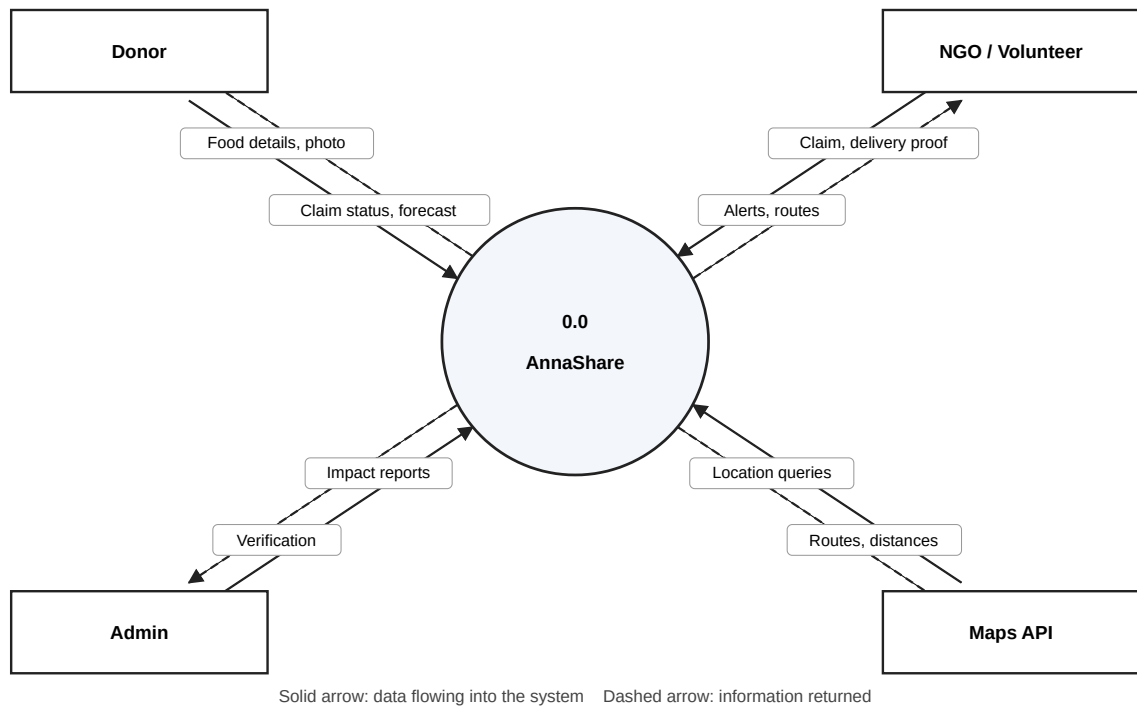


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

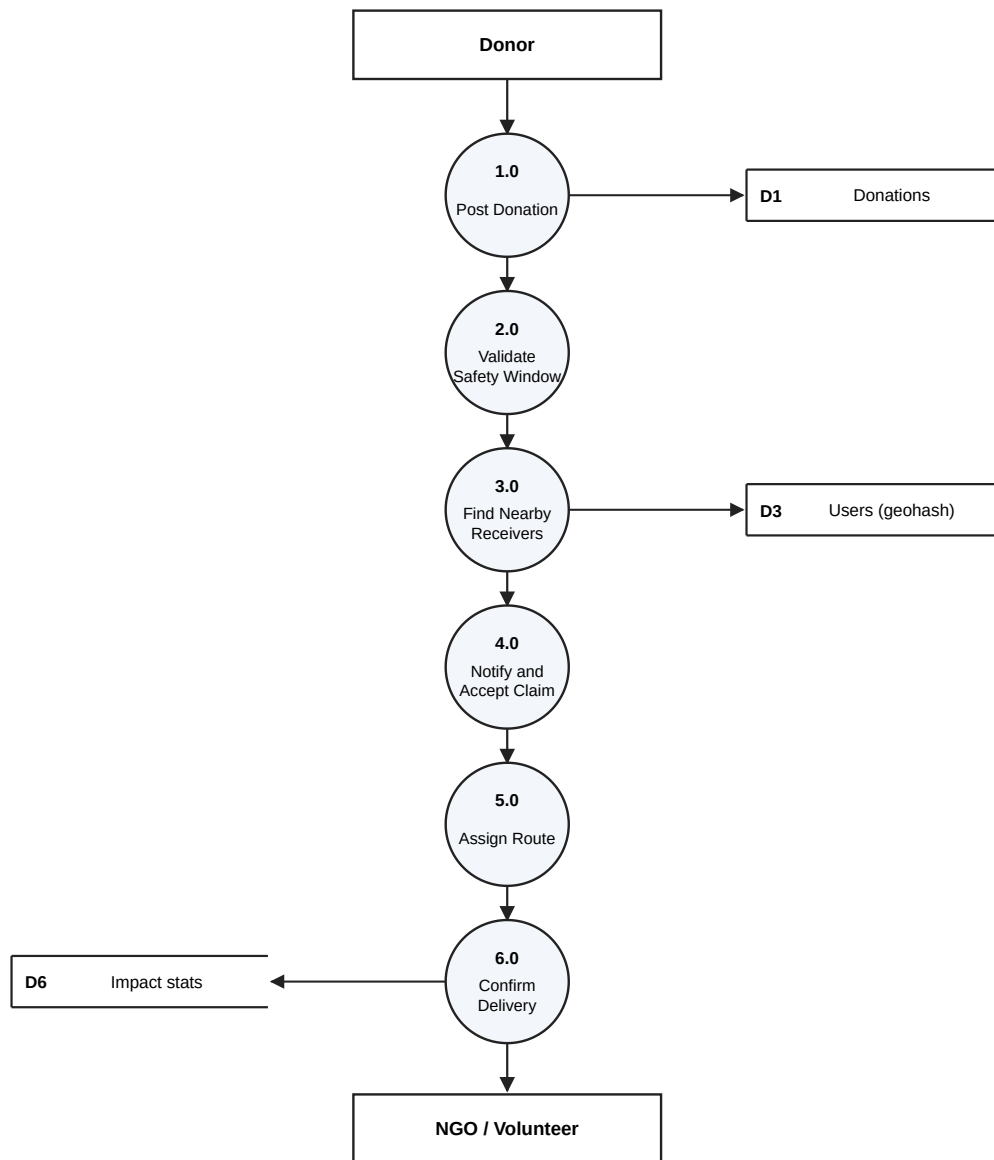


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

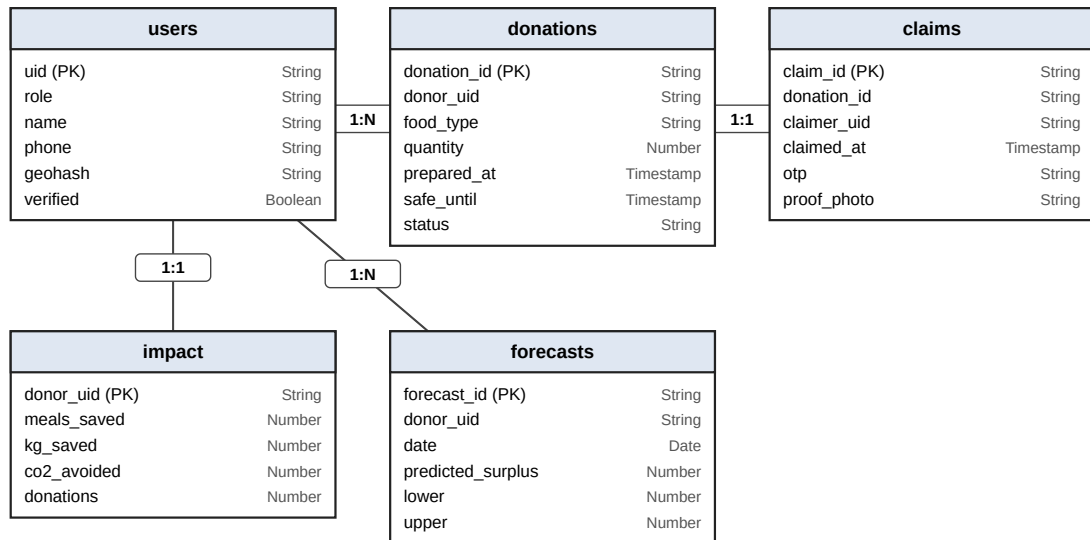


Fig 5: Entity relationship diagram

6.5 Table Design

USERS

FIELD NAME	DATA TYPE	DESCRIPTION
uid	String	Primary key (Firebase UID)
role	String	donor / ngo / volunteer / admin
name	String	Name / organisation
phone	String	Phone
geohash	String	Location geohash
verified	Boolean	NGO verified

DONATIONS

FIELD NAME	DATA TYPE	DESCRIPTION
donation_id	String	Primary key
donor_uid	String	Foreign key to users
food_type	String	veg / non-veg / packaged
quantity	Number	Plates / kg
prepared_at	Timestamp	Preparation time
safe_until	Timestamp	Safe until

status	String	open / claimed / picked / delivered / expired
--------	--------	---

CLAIMS

FIELD NAME	DATA TYPE	DESCRIPTION
claim_id	String	Primary key
donation_id	String	Foreign key to donations
claimer_uid	String	NGO / volunteer
claimed_at	Timestamp	Claim time
otp	String	Delivery OTP
proof_photo	String	Storage URL

IMPACT

FIELD NAME	DATA TYPE	DESCRIPTION
donor_uid	String	Primary key
meals_saved	Number	Total meals
kg_saved	Number	Total kg
co2_avoided	Number	kg CO2e
donations	Number	Count

FORECASTS

FIELD NAME	DATA TYPE	DESCRIPTION
forecast_id	String	Primary key
donor_uid	String	Foreign key to users
date	Date	Forecast date
predicted_surplus	Number	Plates
lower	Number	Lower bound
upper	Number	Upper bound

6.6 Algorithm Design

The matching and forecasting algorithms used in AnnaShare are as follows.

1. **Donor posts quantity, type and expiry time** – A donation document is created in Firestore with status 'open'.
2. **System validates safety window** – Cooked food must have at least 90 minutes remaining before safe_until for pickup and distribution; otherwise the post is rejected with guidance.
3. **Find NGOs/volunteers within radius** – Geohash range queries find verified receivers within 5 km, expanding to 10 km if not claimed in 15 minutes.
4. **Notify and let first responder claim** – Push notifications via FCM; a Firestore transaction ensures only one claim succeeds.
5. **Assign pick-up route using shortest path** – Google Directions API gives the route donor -> receiver; volunteers are ranked by travel time.
6. **Confirm delivery with photo/OTP** – The receiving centre shares a 4-digit OTP; the volunteer enters it and uploads a photo.
7. **Update impact statistics** – Meals saved, kg saved and an estimated CO₂e avoided (using an emission factor per kg of food waste) are updated.
8. **Forecast surplus with time-series model** – Prophet is trained per regular donor on daily surplus history with weekly seasonality and holidays.

PSEUDO CODE

```
ON DONATION_CREATED(d):  
  IF d.safe_until - now < 90 min THEN REJECT(d, "Too close to expiry")  
  r <- 5 km  
  REPEAT  
    receivers <- GEO_QUERY(verified NGOs and volunteers, d.location, r)  
    NOTIFY(receivers, d)  
    WAIT 15 min OR UNTIL claimed  
    r <- r + 5 km  
  UNTIL claimed OR r > 15 km OR now > d.safe_until - 30 min  
  IF NOT claimed THEN MARK_EXPIRED(d)  
  
CLAIM(d, user): TRANSACTION { IF d.status = open THEN d.status <- claimed;  
  d.claimer <- user }
```

6.7 Modules

The modules are grouped by the Donor, NGO/Volunteer and Admin roles.

MODULES

DONOR

1. REGISTER
2. POST SURPLUS FOOD
3. TRACK DONATION
4. VIEW SURPLUS FORECAST

NGO / VOLUNTEER

1. RECEIVE ALERTS AND CLAIM
2. PICKUP AND DELIVER

ADMIN

1. VERIFY NGOS
2. MONITOR IMPACT AND SAFETY

DONOR

REGISTER

Hotels, halls, canteens and individuals register with phone OTP, address and FSSAI licence number (for businesses).

POST SURPLUS FOOD

The donor posts food type, quantity, photo, preparation time and safe-until time; the system validates the safety window.

TRACK DONATION

The donor sees who claimed the food, live pickup status and delivery confirmation.

VIEW SURPLUS FORECAST

Regular donors see the predicted surplus for the next 7 days to plan cooking quantities.

NGO / VOLUNTEER

RECEIVE ALERTS AND CLAIM

Nearby verified NGOs and volunteers receive a push notification and can claim the donation.

PICKUP AND DELIVER

The volunteer follows the route to the donor, collects the food, delivers to the shelter and confirms with photo and OTP.

ADMIN

VERIFY NGOS

Admin verifies NGO registration documents before they can claim food.

MONITOR IMPACT AND SAFETY

Admin views total meals saved, expired posts and complaints, and can block misuse.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

AnnaShare was implemented with a Flutter mobile app and a React web dashboard sharing the same Firebase back end. Cloud Functions written in Node.js handle matching, notifications, claim transactions and impact updates. A Python service using Prophet runs nightly and writes forecasts back to Firestore.

7.2 Implementation

The platform was piloted with two college canteens, one marriage hall and three NGOs over four weeks. Canteen daily surplus records for the previous semester were used to train and evaluate the Prophet forecasting model.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.
- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Post donation	50 plates veg, safe 4 h	Donation open; nearby users notified	Pass
TC02	Unsafe window	Cooked food safe for 45 min	Post rejected with message	Pass
TC03	Simultaneous claim	Two NGOs claim at once	Only first claim succeeds	Pass
TC04	Radius expansion	No claim in 15 min	Notified within 10 km	Pass
TC05	Route	Volunteer accepts pickup	Route displayed with ETA	Pass
TC06	OTP confirmation	Correct OTP + photo	Status delivered; impact updated	Pass
TC07	Wrong OTP	Incorrect OTP	Error; status unchanged	Pass
TC08	Forecast	Canteen with 120 days history	7-day forecast with range	Pass

PERFORMANCE EVALUATION

Pilot results and forecasting accuracy:

Measure	Value
Donations posted (4 weeks)	96
Successfully delivered	89 (92.7%)
Average time to claim	7.5 minutes
Meals saved	4,310
Prophet forecast MAPE (canteen)	14.2% (vs 22.8% naive)

7.4 Source Code

functions/index.js (Firebase Cloud Functions)

```
const functions = require("firebase-functions/v2");
const admin = require("firebase-admin");
const geofire = require("geofire-common");
admin.initializeApp();
const db = admin.firestore();
const MIN_SAFE_MS = 90 * 60 * 1000;

async function nearbyReceivers(center, radiusKm) {
  const bounds = geofire.geohashQueryBounds(center, radiusKm * 1000);
  const snaps = await Promise.all(bounds.map(([s, e]) =>
    db.collection("users").where("role", "in", ["ngo",
"volunteer"]).where("verified", "=", true)
    .orderBy("geohash").startAt(s).endAt(e).get()));
  return snaps.flatMap(s => s.docs).filter(d =>
    geofire.distanceBetween([d.get("lat"), d.get("lng")], center) <= radiusKm);
}

exports.onDonationCreated =
functions.firestore.onDocumentCreated("donations/{id}", async ev => {
  const d = ev.data.data();
  if (d.safe_until.toMillis() - Date.now() < MIN_SAFE_MS) {
    return ev.data.ref.update({ status: "rejected", reason: "Too close to safe-
until time" });
  }
  const users = await nearbyReceivers([d.lat, d.lng], 5);
  const tokens = users.map(u => u.get("fcmToken")).filter(Boolean);
  if (tokens.length) {
    await admin.messaging().sendEachForMulticast({ tokens,
      notification: { title: `Food available: ${d.quantity} ${d.unit}`, body:
`${d.food_type} at ${d.donor_name}. Tap to claim.` },
      data: { donationId: ev.params.id } });
  }
  return ev.data.ref.update({ notified_radius_km: 5, notified_at:
admin.firestore.FieldValue.serverTimestamp() });
});

exports.claimDonation = functions.https.onCall(async req => {
  const uid = req.auth.uid, id = req.data.donationId;
  return db.runTransaction(async tx => {
    const ref = db.doc(`donations/${id}`); const snap = await tx.get(ref);
    if (snap.get("status") !== "open") throw new
functions.https.HttpsError("failed-precondition", "Already claimed");
    const otp = String(Math.floor(1000 + Math.random() * 9000));
    tx.update(ref, { status: "claimed", claimer_uid: uid, claimed_at:
admin.firestore.FieldValue.serverTimestamp() });
    tx.set(db.doc(`claims/${id}`), { donation_id: id, claimer_uid: uid, otp });
    return { ok: true };
  });
});

exports.confirmDelivery = functions.https.onCall(async req => {
  const { donationId, otp, photoUrl } = req.data;
  const claim = await db.doc(`claims/${donationId}`).get();
  if (claim.get("otp") !== otp) throw new
functions.https.HttpsError("permission-denied", "Wrong OTP");
  const don = (await db.doc(`donations/${donationId}`).get()).data();
```

```

const kg = don.unit === "kg" ? don.quantity : don.quantity * 0.4;
await db.doc(`donations/${donationId}`).update({ status: "delivered",
proof_photo: photoUrl });
await db.doc(`impact/${don.donor_uid}`).set({
  meals_saved: admin.firestore.FieldValue.increment(Math.round(kg / 0.4)),
  kg_saved: admin.firestore.FieldValue.increment(kg),
  co2_avoided: admin.firestore.FieldValue.increment(kg * 2.5),
  donations: admin.firestore.FieldValue.increment(1) }, { merge: true });
return { ok: true };
});

```

forecast/forecast.py

```

import pandas as pd
from prophet import Prophet
from google.cloud import firestore

db = firestore.Client()
holidays = pd.read_csv("tn_holidays.csv")          # columns: holiday, ds

def forecast_donor(uid, days=7):
    hist = pd.DataFrame([d.to_dict() for d in
db.collection("surplus_log").where("donor_uid", "==", uid).stream()])
    if len(hist) < 60:
        return None
    df = hist.rename(columns={"date": "ds", "plates": "y"})[["ds", "y"]]
    m = Prophet(weekly_seasonality=True, yearly_seasonality=False,
holidays=holidays)
    m.add_regressor("exam_week") if "exam_week" in hist else None
    m.fit(df)
    fut = m.make_future_dataframe(periods=days)
    fc = m.predict(fut).tail(days)
    for r in fc.itertuples():
        db.collection("forecasts").add({"donor_uid": uid, "date":
r.ds.date().isoformat(),
                                "predicted_surplus": max(round(r.yhat),
0),
                                "lower": max(round(r.yhat_lower), 0),
"upper": round(r.yhat_upper)})
    return fc

if __name__ == "__main__":
    for u in db.collection("users").where("role", "==",
"donor").where("regular", "==", True).stream():
        forecast_donor(u.id)

```

7.5 Screen Layout

SCREENSHOTS:

POST SURPLUS FOOD (DONOR)

AnnaShare

[Donate](#)[Nearby Food](#)[Pickups](#)[Impact](#)[Forecast](#)

Donate surplus food

Food

Veg meals - rice, sambar, poriyal

Quantity

60 plates

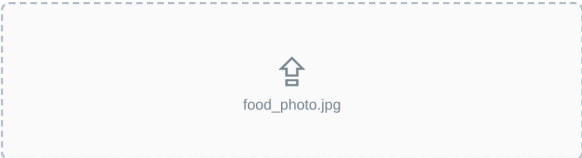
Prepared at

12:30 PM

Safe until

5:30 PM

Photo

food_photo.jpg

Safety checklist

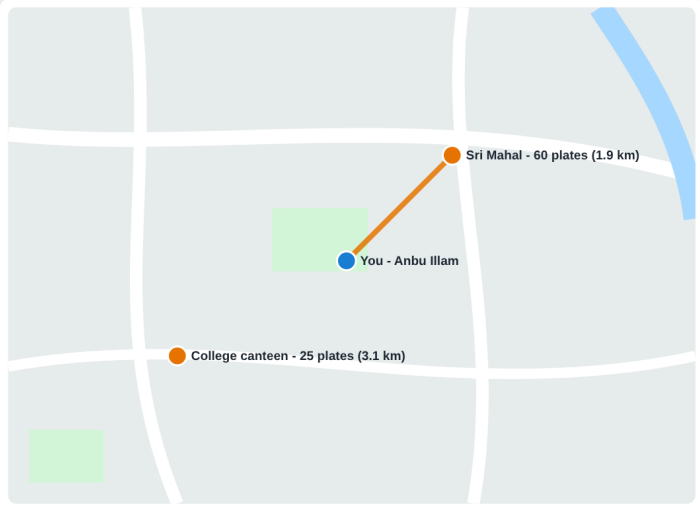
- Food covered and kept hygienically.
- Not left over from plates.
- Prepared within the last few hours.
- Mention allergens if any (nuts, milk).

NEARBY FOOD ALERT (NGO)

AnnaShare

[Donate](#)[Nearby Food](#)[Pickups](#)[Impact](#)[Forecast](#)

Food available near you



Donor	Sri Mahal Wedding Hall
Food	Veg meals, 60 plates
Safe until	5:30 PM (4 h left)
Distance	1.9 km - 8 min
Action	[CLAIM PICKUP]

PICKUP TRACKING

https://annashare.app/

AnnaShare

DonateNearby FoodPickupsImpactForecast

Today's pickups

Donor	Food	Claimed by	Status	Safe until
Sri Mahal	60 plates veg	Anbu Illam	PICKED UP	5:30 PM
College canteen	25 plates	Volunteer Karthik	ON THE WAY	4:00 PM
Hotel Annapoorna	8 kg rice	Karunai Trust	DELIVERED	3:00 PM
Household	10 chapatis	-	EXPIRED	1:00 PM

IMPACT AND FORECAST

https://annashare.app/

AnnaShare

DonateNearby FoodPickupsImpactForecast

Impact - Sri Mahal / Forecast - College Canteen

Meals saved (city)
4,310

Food saved
1,724 kg

CO2e avoided
4.3 t

Active NGOs
18

Canteen surplus forecast - next 7 days (plates)

Day	Surplus (plates)
Mon	22
Tue	18
Wed	24
Thu	20
Fri	34
Sat	10
Sun	8

Suggestion

Day	Advice
Fri	Cook ~30 plates less (event cancelled)
Sat	Half-day: reduce rice by 20%
Sun	Hostel only: reduce by 40%

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Surplus Food Donation and Waste Reduction Platform connects donors with nearby NGOs and volunteers in real time, enforcing food safety windows and confirming deliveries. In the four-week pilot, 92.7% of posted donations were delivered with an average claim time of 7.5 minutes, saving over 4,000 meals.

The Prophet-based surplus forecast reduced forecasting error for the college canteen from 22.8% to 14.2%, helping kitchens prepare closer to actual demand. The platform therefore tackles food waste both by redistribution and by prevention at the source.

8.2 Future Enhancement

- Temperature-logging insulated boxes with IoT sensors for cold chain.
- Integration with FSSAI's Save Food Share Food network.
- Gamification and recognition for frequent donors and volunteers.
- Multi-stop route optimisation for volunteers collecting from several donors.
- Tamil voice interface for small donors.

CHAPTER IX

BIBLIOGRAPHY

1. United Nations Environment Programme, "Food Waste Index Report 2021", Nairobi, 2021.
2. Food Safety and Standards Authority of India, "Food Safety and Standards (Recovery and Distribution of Surplus Food) Regulations", 2019.
3. Taylor, S. J., and Letham, B., "Forecasting at Scale", The American Statistician, Vol. 72, No. 1, pp. 37-45, 2018.
4. Box, G. E. P., and Jenkins, G. M., Time Series Analysis: Forecasting and Control, Holden-Day, 1970.
5. Harvey, J., Smith, A., Goulding, J., and Branco Illodo, I., "Food sharing, redistribution, and waste reduction via mobile applications: A social network analysis", Industrial Marketing Management, Vol. 88, pp. 437-448, 2020.
6. Dijkstra, E. W., "A note on two problems in connexion with graphs", Numerische Mathematik, Vol. 1, pp. 269-271, 1959.

WEBSITE REFERENCE

1. <https://firebase.google.com/docs>
2. <https://docs.flutter.dev>
3. <https://facebook.github.io/prophet>
4. <https://fssai.gov.in>